

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

Course Project Report

Name of the Project

IoT-Based Smart Energy Monitoring and Electrical Safety Control
System Using Multi-Sensor Integration

Team Members: Danasri M S

Rishikeshwar A D

Aadithya S

Abstract

The increasing demand for electricity and the rising risks associated with electrical systems, such as gas leakage, overheating, over-voltage, and unauthorized access, highlight the need for a smart monitoring solution. Traditional systems lack real-time monitoring and integrated safety mechanisms, making them inefficient in preventing hazards. This project proposes an IoT-based smart energy monitoring and electrical safety control system using multi-sensor integration to address these challenges.

The system utilizes various sensors, including gas (MQ4), temperature (LM35), current (ACS712), voltage, vibration, and PIR motion sensors, to continuously monitor environmental and electrical parameters. An Arduino UNO processes the sensor data, while the ESP8266 module enables wireless communication and hosts a web-based dashboard for real-time monitoring. The system is designed to trigger alerts and control electrical loads automatically when abnormal conditions are detected.

The technologies used in this project include embedded systems programming (Arduino C/C++), IoT communication using ESP8266, and web server-based data visualization. The results demonstrate effective real-time monitoring, timely alert generation, and improved electrical safety. This system provides a low-cost, scalable solution suitable for smart homes and industrial safety applications.

1. Introduction

Background of the Problem

With the rapid increase in electricity consumption in residential and industrial environments, the risk of electrical hazards such as gas leakage, overheating, over-voltage, and equipment failure has significantly increased. Traditional electrical systems lack continuous monitoring and early warning mechanisms, making it difficult to detect abnormal conditions in real time. This often leads to accidents, energy wastage, and damage to appliances. Therefore, there is a growing need for intelligent systems that can ensure both safety and efficient energy usage.

Need for IoE-Based Solution

The Internet of Everything (IoE) connects devices, sensors, and systems to enable seamless data exchange and smart decision-making. An IoE-based solution allows real-time monitoring of multiple parameters through interconnected sensors and smart devices. By integrating IoT technologies such as Arduino and ESP8266, the system can collect, process, and transmit data over the internet. This enables remote monitoring, instant alerts, and automated control, significantly improving safety, reliability, and efficiency compared to traditional systems.

Scope of the Project

This project focuses on developing a smart energy monitoring and electrical safety system using multiple sensors. It monitors parameters such as gas leakage, temperature, current, voltage, vibration, and human motion. The system provides real-time data through a web-based dashboard and triggers alerts when abnormal conditions are detected. The solution is designed to be low-cost, scalable, and suitable for applications in smart homes, industries, and commercial buildings. Future enhancements can include cloud integration, mobile applications, and advanced analytics for predictive maintenance.

2. Problem Statement

Electrical systems in homes and industries are increasingly prone to hazards such as gas leakage, overheating, over-voltage, excessive current flow, and unauthorized access. Existing traditional systems lack real-time monitoring, integrated multi-parameter sensing, and automated safety control mechanisms. As a result, abnormal conditions often go undetected, leading to potential accidents, equipment damage, energy wastage, and safety risks.

There is a need for a reliable and intelligent system that can continuously monitor multiple environmental and electrical parameters, detect faults in real time, and provide immediate alerts along with automatic control actions. The challenge is to design a cost-effective and scalable solution that integrates various sensors and enables remote monitoring through IoT technology.

3. Objectives

- **Objective 1:**

To design and develop an IoT-based system for real-time monitoring of electrical and environmental parameters such as current, voltage, temperature, gas leakage, vibration, and motion.

- **Objective 2:**

To detect abnormal conditions using predefined threshold values and trigger alerts along with automatic safety control actions (e.g., buzzer and relay operation).

- **Objective 3:**

To implement a web-based dashboard using ESP8266 for remote monitoring and visualization of sensor data in real time.

- **Objective 4:**

To develop a low-cost, scalable, and reliable solution suitable for smart homes and industrial safety applications.

4. Literature Survey

1. Existing Systems or Related Research

Paper 1: Smart Home Energy Management System (Sureshkumar et al., 2024)

This system focuses on monitoring household electricity and water consumption using IoT-based smart switch boxes and multi-sensor data fusion. It enables real-time monitoring through web or mobile applications and aims at energy optimization, cost reduction, and sustainability.

However, it mainly concentrates on energy management and does not provide comprehensive electrical safety features.

Paper 2: IoT-Based Smart Home and Security System (Suriya Prakash et al., 2025)

This work presents a home automation and security system using ESP32 and multiple sensors such as gas, fire, rain, and IR sensors. It includes hazard detection, QR-based access control, and mobile notifications.

While it enhances home security, it does not focus on energy monitoring or electrical parameter analysis.

Paper 3: IoT System for Smart City Emergency Management (Fedele & Merenda, 2020)

This paper introduces a large-scale IoT-based wireless sensor network for crowd monitoring and emergency response in smart cities. It integrates neural networks and real-time data processing. The system is designed for smart cities and not suitable for household-level electrical monitoring.

Paper 4: IoT-Based Agricultural Monitoring System (Kar et al., 2023)

This system uses Raspberry Pi and multi-sensors to monitor environmental parameters such as temperature, humidity, and pressure for agricultural applications. It focuses on environmental monitoring and does not address electrical safety or energy management.

2. Technologies Used in Similar Projects

Common technologies used across existing systems include:

- **Microcontrollers:** Arduino, ESP32, Raspberry Pi
- **Communication:** Wi-Fi, MQTT, NFC
- **Platforms:** Web servers, Firebase, Blynk
- **Sensors:** Gas, temperature, humidity, pressure, IR
- **Algorithms:** Neural networks (for advanced systems)
- **Interfaces:** Mobile apps and web dashboards

These systems typically focus on:

- Energy management
- Home security
- Environmental monitoring
- Smart city applications

3. Limitations of Existing Solutions

- Focus on **single functionality** (only energy or only security)
- Lack of **integrated safety + energy monitoring**
- Limited **electrical parameter tracking** (voltage/current not included)

- High complexity or cost in large-scale systems
- Lack of **real-time alert classification systems**

4. How Our Project Improves or Differs

1. Integrated Energy and Safety Monitoring

- Combines energy monitoring and safety detection in one system:
 - Voltage & current monitoring
 - Gas leakage detection
 - Temperature sensing
 - Motion and vibration detection

2. Dual Microcontroller Architecture

- Arduino UNO → Sensor data processing
- ESP8266 → IoT communication & dashboard
- Improves modularity and system reliability

3. IoT Dashboard Using Blynk

- Displays real-time sensor data on mobile application
- Provides easy remote monitoring from anywhere
- Supports live updates through Wi-Fi connectivity
- User-friendly interface for quick visualization

4. Electrical Parameter Monitoring

- Uses **ACS712 (current sensor)**

- Uses **Voltage detection sensor**
- Helps in overload detection and fire prevention

5. Low-Cost and Practical Solution

- Uses affordable components
- Easy to implement
- Suitable for homes, labs, and small industries

5. Proposed System Overview

Overall Working Concept

The proposed system is an IoT-based smart energy monitoring and electrical safety system that integrates multiple sensors to continuously monitor environmental and electrical parameters. Sensors such as gas, temperature, current, voltage, vibration, and motion are connected to the Arduino UNO, which processes the collected data.

The processed data is then transmitted to the NodeMCU (ESP8266), which enables wireless communication over Wi-Fi. The system sends real-time data to the Blynk application, allowing users to monitor parameters remotely through a mobile dashboard.

When any parameter exceeds predefined threshold values (such as high temperature, gas leakage, or overcurrent), the system automatically triggers alerts using a buzzer and activates safety mechanisms like relay control to prevent damage or hazards. This ensures real-time monitoring, quick response, and improved electrical safety.

Key Features of the System

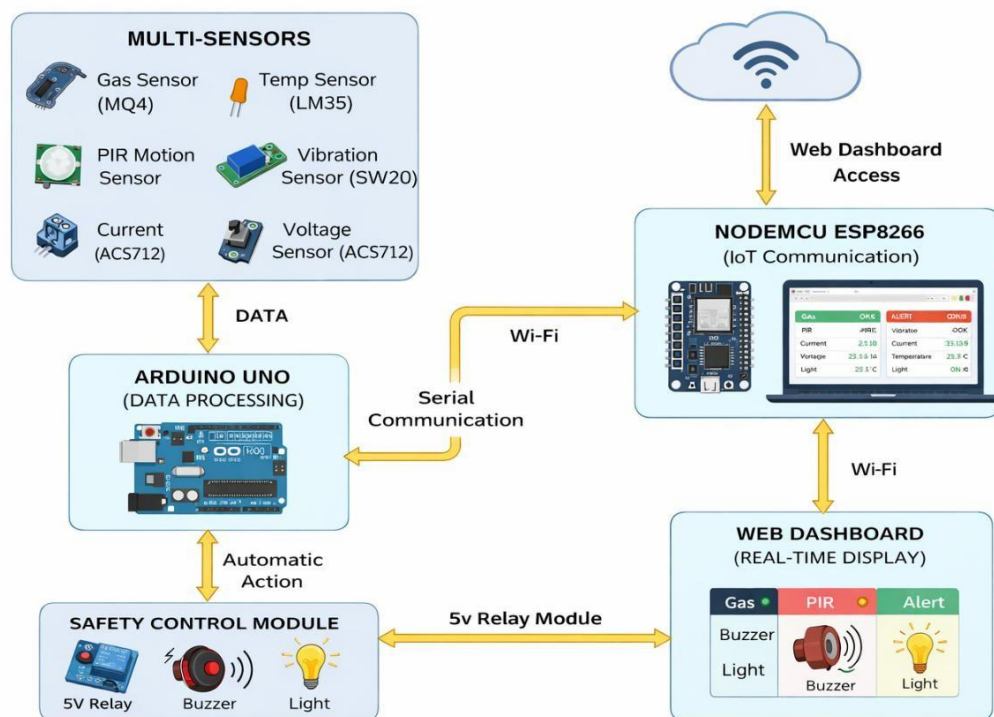
- **Multi-Sensor Integration:**
Combines gas, temperature, current, voltage, vibration, and motion sensors in one system

- **Real-Time Monitoring:**
Continuously tracks system parameters and updates data instantly
- **IoT-Based Remote Access:**
Allows users to monitor data remotely using the Blynk mobile application
- **Automatic Safety Control:**
Activates buzzer and relay when abnormal conditions are detected
- **Electrical Parameter Monitoring:**
Detects over-voltage and over-current conditions

6. System Architecture

1. Block Diagram of the System

Represents the overall architecture. The functional block diagram of your system can be represented as:



IoT-Based Smart Energy Monitoring and Electrical Safety Control System Using Multi-Sensor Integration

2. Module-Level Description

1. Sensor Module

- Includes:
 - Gas sensor (MQ4)
 - Temperature sensor (LM35)
 - Current sensor (ACS712)
 - Voltage sensor
 - Vibration sensor (SW420)
 - PIR motion sensor
- Responsible for collecting real-time environmental and electrical data

2. Processing Module (Arduino UNO)

- Reads sensor values using analog/digital pins
- Converts raw data into meaningful values
- Compares values with threshold limits
- Controls output devices (buzzer, relay)

3. Communication Module (ESP8266 NodeMCU)

- Receives data from Arduino via serial communication
- Connects to Wi-Fi network
- Sends data to Blynk platform
- Enables remote monitoring

4. Output & Control Module

- **Buzzer:** Provides alert during abnormal conditions
- **Relay Module:** Controls electrical load (ON/OFF)

5. User Interface Module

- Mobile dashboard using Blynk
- Displays real-time sensor data
- Allows remote monitoring

Data Flow Explanation

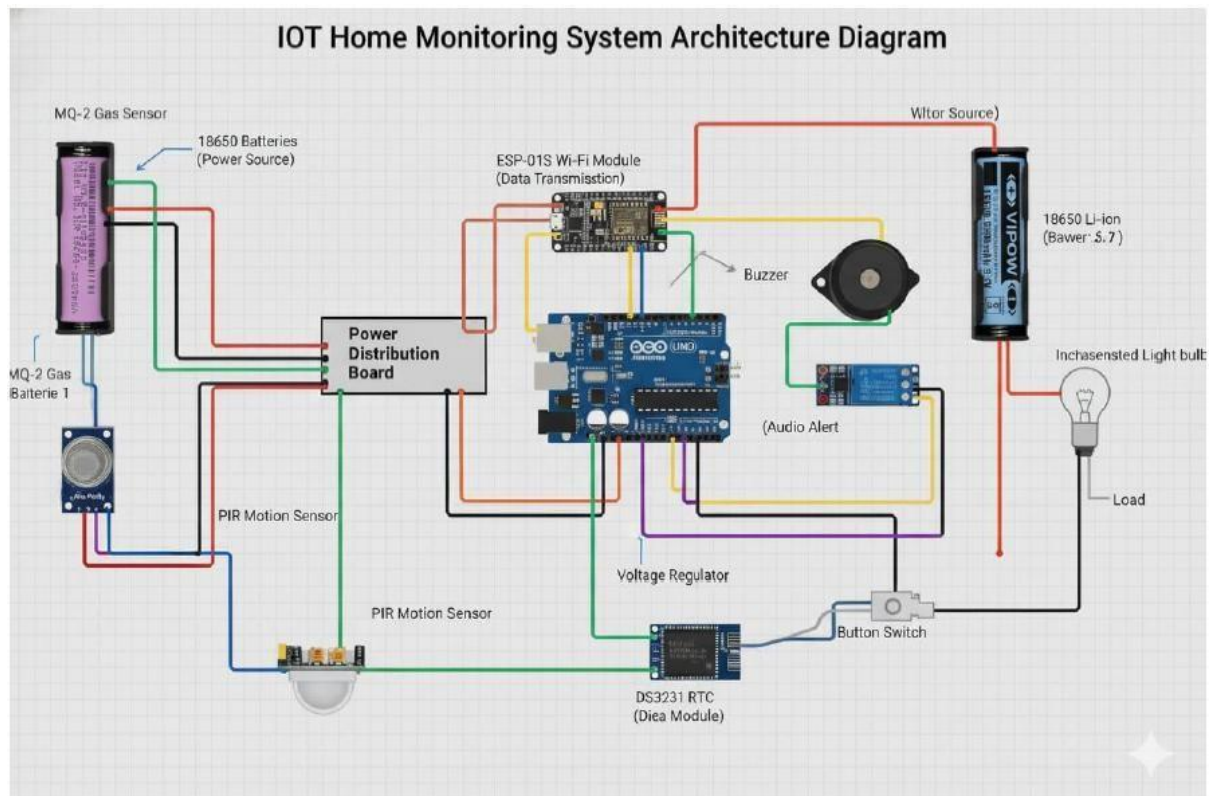
1. Sensors collect environmental and electrical data continuously.
2. Arduino UNO reads and processes the sensor data.
3. If thresholds are exceeded → Arduino triggers buzzer/relay.
4. Processed data is sent to ESP8266 via serial communication.
5. ESP8266 transmits data through Wi-Fi.
6. Data is displayed on the Blynk mobile dashboard.
7. User can monitor system status remotely in real time.

7. Hardware Design

Components Used

Component	Model/Type	Purpose
Microcontroller	Arduino UNO	Reads sensor data, processes signals, checks threshold conditions, and controls relay/buzzer/light.
Wi-Fi Module	NodeMCU ESP8266 (CP2102)	Provides IoT connectivity, hosts web server, and sends real-time sensor data to dashboard.
Gas Sensor	MQ4	Detects LPG/methane gas leakage and triggers safety alerts.
Temperature Sensor	LM35	Measures ambient temperature to detect overheating conditions.
Current Sensor	ACS712 (5A)	Measures current flowing through load to detect overload conditions.
Voltage Detection Sensor	25V Voltage Sensor Module	Monitors voltage levels and identifies abnormal voltage fluctuations.
Vibration Sensor	SW420	Detects abnormal vibrations in electrical equipment.
Motion Sensor	PIR Sensor HC-SR501	Detects human motion for safety monitoring.
Relay Module	5V Relay Module	Automatically controls load (ON/OFF) during abnormal conditions.
Buzzer	5V Active Buzzer	Provides audible alert during hazard detection.
Light Indicator	12V DC LED Bulb (0.5W)	Visual indication of alert or system status.
Power Supply	3 × 2500mAh Li-ion Batteries	Supplies power to the system.
Breadboard	Small Breadboard	Used for circuit prototyping and connections.
Jumper Wires	Male-to-Male / Male-to-Female	Electrical connections between components.
Switch	Small Toggle Switch	Manual ON/OFF control of system modules.

Circuit Diagram:



Hardware Setup Description

The hardware setup of the proposed system integrates multiple sensors and modules with the Arduino UNO and ESP8266 to enable real-time monitoring and control. All sensors are connected to the appropriate analog and digital pins of the Arduino UNO, which acts as the central processing unit.

The gas sensor (MQ4), temperature sensor (LM35), current sensor (ACS712), and voltage sensor are connected to analog input pins to measure environmental and electrical parameters. The PIR motion sensor and vibration sensor (SW420) are connected to digital pins to detect motion and disturbances.

The Arduino processes the sensor data and communicates with the NodeMCU (ESP8266) using serial communication. The ESP8266 is responsible for Wi-Fi connectivity and sends the processed data to the Blynk application for remote monitoring.

A relay module is connected to control the electrical load (such as an LED bulb), allowing automatic switching during abnormal conditions. A buzzer is used to provide audible alerts when thresholds are exceeded. The entire system is powered using a rechargeable Li-ion battery, ensuring portability and continuous operation.

8. Software Design

Programming Environment

- **Arduino IDE** is used for writing and uploading code to Arduino UNO and ESP8266
- Programming language: **Embedded C / Arduino C++**
- Serial communication is used for data exchange between Arduino and ESP8266

Libraries Used

- `<SoftwareSerial.h>` – Enables serial communication between Arduino and ESP8266
- `<ESP8266WiFi.h>` – Used for Wi-Fi connectivity
- `<ESP8266WebServer.h>` – Used for creating a local web server (if applicable)
- Standard Arduino libraries for sensor interfacing

Communication Protocol

- **Wi-Fi communication** is used via ESP8266 module
- Data is transmitted wirelessly to the IoT platform
- Serial communication (UART) is used between Arduino and ESP8266

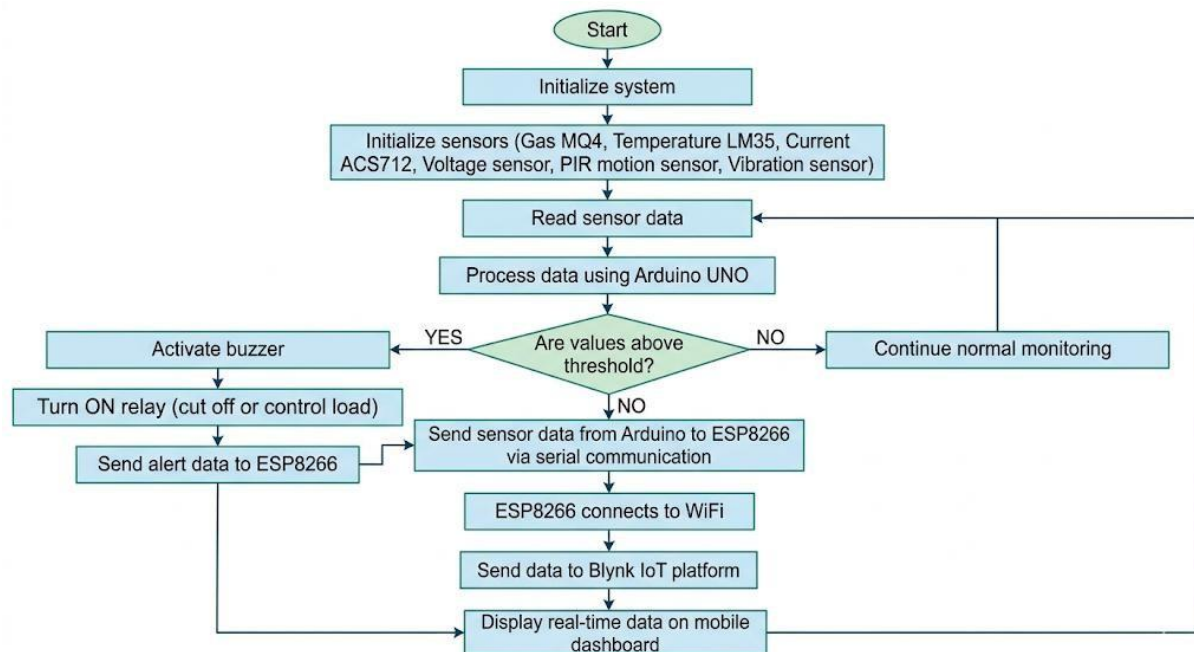
Cloud Platform / Server Details

- The system uses the Blynk platform for:
 - Real-time data monitoring
 - Remote access via mobile application
 - Visualization of sensor values
- ESP8266 sends data over Wi-Fi to the Blynk cloud/server

Algorithm Steps

1. Start the system
2. Initialize sensors and communication modules
3. Read sensor values (gas, temperature, voltage, current, motion, vibration)
4. Compare values with predefined threshold limits
5. If any value exceeds threshold:
 - Activate buzzer
 - Turn ON relay (safety action)
6. Send sensor data to ESP8266
7. ESP8266 transmits data to Blynk
8. Display data on mobile dashboard
9. Repeat continuously

Flow Chart:



9. Implementation

1. Hardware Integration

- All sensors (MQ4, LM35, ACS712, voltage sensor, PIR, SW420) are connected to the **Arduino UNO** using appropriate analog and digital pins.
- The **ACS712 and voltage sensor** are connected to analog inputs for electrical parameter measurement.
- The **PIR and vibration sensors** are connected to digital pins for motion detection.
- The **relay module** is connected to control the electrical load (LED bulb).
- The **buzzer** is connected to provide alerts during abnormal conditions.
- The **ESP8266 (NodeMCU)** is connected to Arduino via **serial communication (TX/RX)**.
- A **Li-ion battery** is used to power the system.

2. Code Development

- The program is written using **Arduino IDE (Embedded C / C++)**.
- Sensor values are read using:
 - `analogRead()` for analog sensors
 - `digitalRead()` for digital sensors
- Sensor data is processed and converted into meaningful values (temperature, voltage, current).
- Threshold conditions are defined:
 - Gas level
 - Temperature ($> 40\text{--}45^{\circ}\text{C}$)
 - Voltage/current limits
- If thresholds are exceeded:
 - **Buzzer is activated**
 - **Relay is triggered**
- Sensor data is sent to ESP8266 using **SoftwareSerial communication**.
- ESP8266 code:
 - Connects to Wi-Fi network
 - Sends data to IoT platform
 - Updates dashboard continuously

3. Cloud / Dashboard Setup

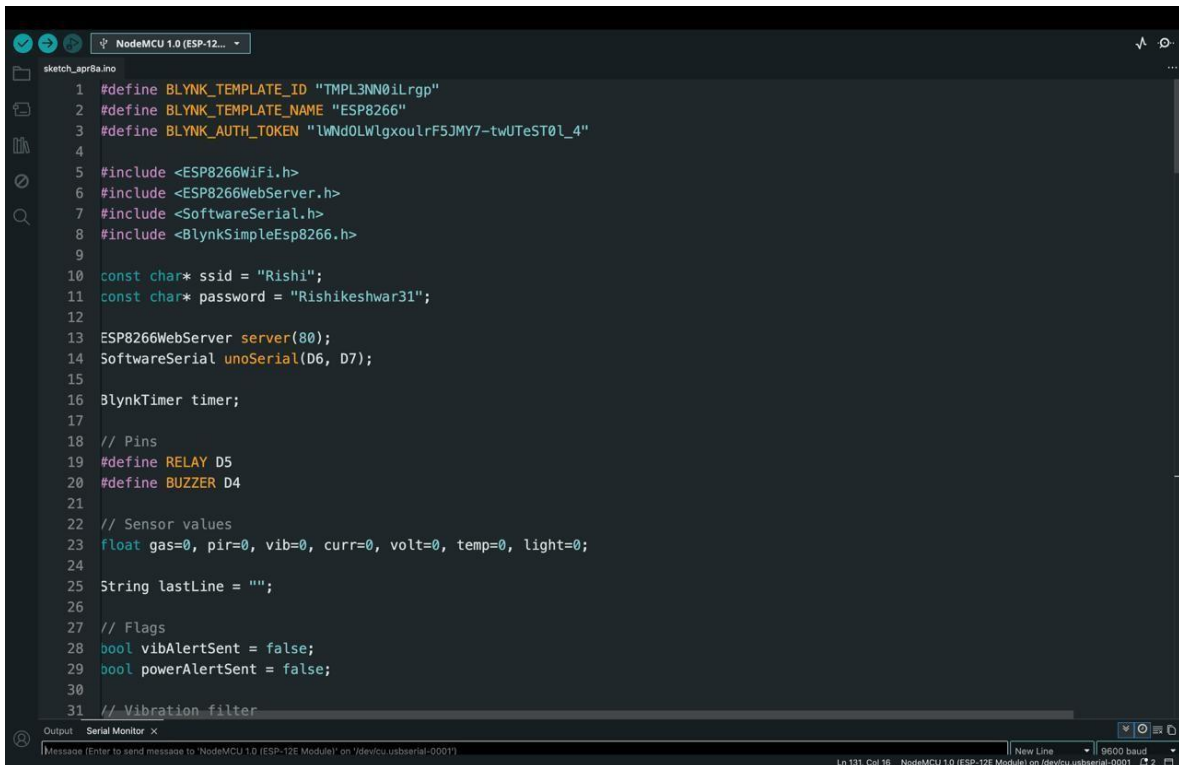
- The system uses the Blynk platform for IoT monitoring.

Steps:

1. Create an account in Blynk mobile application
2. Create a new project and select ESP8266 as the device

3. Obtain the **authentication token**
4. Add widgets (e.g., gauges, labels) for:
 - Gas
 - Temperature
 - Voltage
 - Current
 - Motion
5. Configure virtual pins for each sensor
6. Connect ESP8266 to Wi-Fi using credentials
7. Send sensor data to Blynk using the auth token
8. Monitor real-time data on mobile dashboard

Code screenshot:

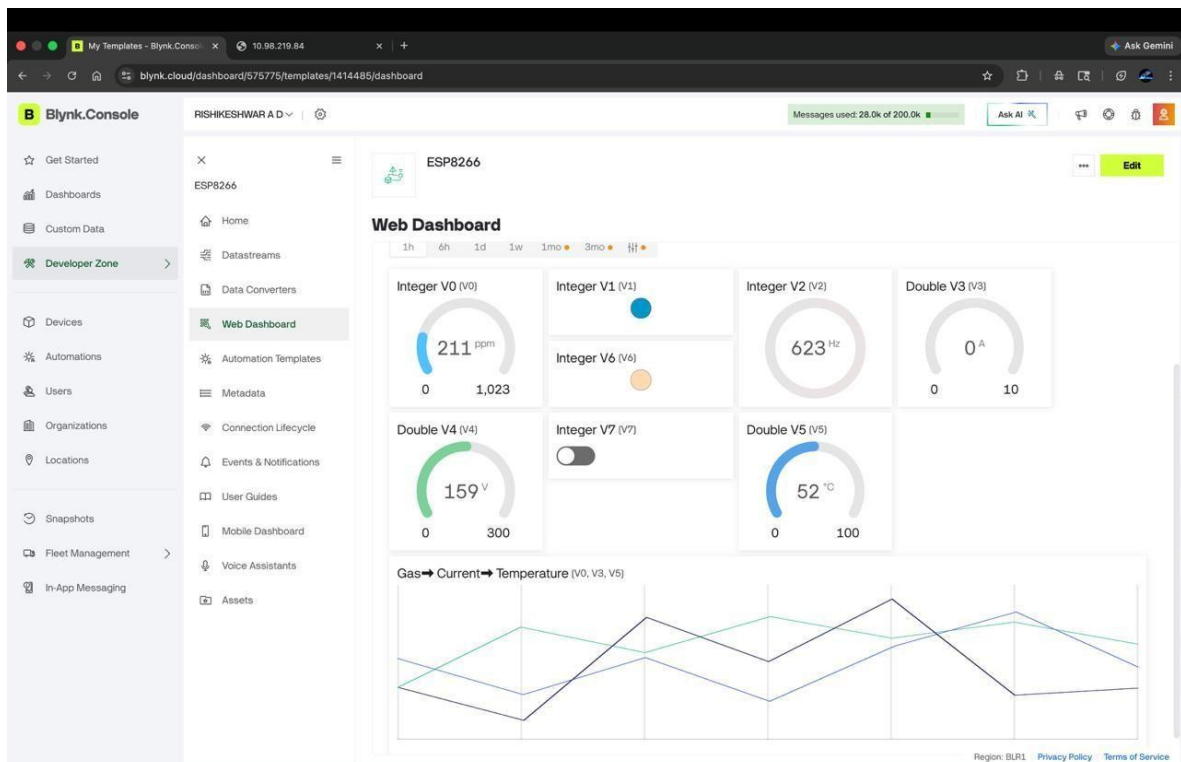


```
1 #define BLYNK_TEMPLATE_ID "TMPL3NN0iLrgp"
2 #define BLYNK_TEMPLATE_NAME "ESP8266"
3 #define BLYNK_AUTH_TOKEN "lwNd0LWlgxoulrF5JMY7-twUTeST0l_4"
4
5 #include <ESP8266WiFi.h>
6 #include <ESP8266WebServer.h>
7 #include <SoftwareSerial.h>
8 #include <BlynkSimpleEsp8266.h>
9
10 const char* ssid = "Rishi";
11 const char* password = "Rishikeshwar31";
12
13 ESP8266WebServer server(80);
14 SoftwareSerial unoSerial(D6, D7);
15
16 BlynkTimer timer;
17
18 // Pins
19 #define RELAY D5
20 #define BUZZER D4
21
22 // Sensor values
23 float gas=0, pir=0, vib=0, curr=0, volt=0, temp=0, light=0;
24
25 String lastLine = "";
26
27 // Flags
28 bool vibAlertSent = false;
29 bool powerAlertSent = false;
30
31 // Vibration filter
```

10. Results & Analysis

```
NodeMCU 1.0 (ESP-12E)
sketch_apr8a.ino
1 #define BLYNK_TEMPLATE_ID "TMPL3NN0iLrgp"
2 #define BLYNK_TEMPLATE_NAME "ESP8266"
3 #define BLYNK_AUTH_TOKEN "lwNd0LWlgxouLrF5JMY7-twUTeSt0l_4"
4
5 #include <ESP8266WiFi.h>
6 #include <ESP8266WebServer.h>
7 #include <SoftwareSerial.h>
8 #include <BlynkSimpleEsp8266.h>
9
10 const char* ssid = "Rishi";
11 const char* password = "Rishikeshwar31";
12
13 ESP8266WebServer server(80);

Output Serial Monitor x
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on '/dev/cu.usbserial-0001') New Line 9600 baud
22:03:30.367 -> Connected:
22:03:36.360 -> 10.98.219.84
22:03:36.623 -> RAW: GAS:24,PIR:0,VIB:120,CURR:18.04,VOLT:3.91,TEMP:38.6,LIGHT:0
22:03:36.687 -> VIB: 120.00
22:03:37.182 -> RAW: GAS:24,PIR:0,VIB:120,CURR:18.04,VOLT:3.91,TEMP:39.1,LIGHT:0
22:03:38.269 -> RAW: GAS:24,PIR:0,VIB:120,CURR:18.04,VOLT:3.91,TEMP:39.1,LIGHT:0
22:03:38.334 -> VIB: 120.00
22:03:39.322 -> RAW: GAS:24,PIR:0,VIB:120,CURR:18.04,VOLT:3.91,TEMP:39.6,LIGHT:0
22:03:39.386 -> VIB: 120.00
22:03:40.410 -> RAW: GAS:24,PIR:0,VIB:120,CURR:18.04,VOLT:3.91,TEMP:38.6,LIGHT:0
22:03:40.475 -> VIB: 120.00
22:03:41.465 -> RAW: GAS:24,PIR:0,VIB:120,CURR:18.04,VOLT:3.91,TEMP:39.6,LIGHT:0
22:03:41.530 -> VIB: 120.00
22:03:42.552 -> RAW: GAS:24,PIR:0,VIB:120,CURR:18.04,VOLT:3.91,TEMP:39.1,LIGHT:0
22:03:42.619 -> VIB: 120.00
22:03:43.606 -> RAW: GAS:23,PIR:0,VIB:120,CURR:18.04,VOLT:3.91,TEMP:39.1,LIGHT:0
22:03:43.670 -> VIB: 120.00
22:03:44.692 -> RAW: GAS:24,PIR:0,VIB:120,CURR:18.04,VOLT:3.91,TEMP:39.6,LIGHT:0
22:03:44.757 -> VIB: 120.00
22:03:46.043 -> RAW: GAS:23,PIR:0,VIB:120,CURR:18.04,VOLT:3.91,TEMP:39.1,LIGHT:0
22:03:46.107 -> VIB: 120.00
22:03:47.030 -> RAW: GAS:24,PIR:0,VIB:120,CURR:18.04,VOLT:3.91,TEMP:39.1,LIGHT:0
22:03:47.094 -> VIB: 120.00
22:03:48.017 -> RAW: GAS:23,PIR:0,VIB:119,CURR:18.04,VOLT:3.91,TEMP:38.6,LIGHT:0
22:03:48.081 -> VIB: 119.00
22:03:49.038 -> RAW: GAS:24,PIR:0,VIB:119,CURR:17.99,VOLT:3.91,TEMP:39.1,LIGHT:0
22:03:49.103 -> VIB: 119.00
22:03:50.027 -> RAW: GAS:24,PIR:0,VIB:119,CURR:17.99,VOLT:3.91,TEMP:38.6,LIGHT:0
22:03:50.091 -> VIB: 119.00
```



11. Challenges Faced

- **Sensor Calibration Issues:** Obtaining accurate readings from sensors like ACS712 (current) and MQ4 (gas) required proper calibration and adjustment of offset values.
- **Noise and Fluctuations in Data:** Sensor readings were sometimes unstable due to electrical noise, especially in voltage and current measurements.
- **Serial Communication Errors:** Data transmission between Arduino UNO and ESP8266 occasionally resulted in incomplete or misaligned data due to baud rate or timing issues.
- **Wi-Fi Connectivity Problems:** The ESP8266 faced intermittent connectivity issues, affecting real-time data transmission to the Blynk platform.
- **Power Supply Limitations:** Maintaining stable power using Li-ion batteries was challenging when multiple sensors and modules were active.
- **Threshold Value Tuning:** Setting appropriate threshold values for parameters like gas, temperature, and voltage required multiple trials to avoid false alerts.
- **Integration Complexity:** Combining multiple sensors, microcontrollers, and IoT communication into a single system increased system complexity and debugging effort.

12. Applications

The proposed IoT-based smart energy monitoring and electrical safety system can be applied in various real-world scenarios to improve safety, efficiency, and automation.

Real-World Use Cases

- **Smart Homes:** Monitors energy usage, detects gas leaks, and ensures safety through automatic alerts and control mechanisms.

- **Industrial Safety Systems:**Detects abnormal conditions such as overheating, gas leakage, and equipment vibration to prevent accidents and machinery damage.
- **Commercial Buildings:**Provides centralized monitoring of electrical parameters and enhances safety in offices, malls, and public spaces.
- **Energy Management Systems:**Helps in tracking power consumption and identifying energy wastage for efficient usage.
- **Laboratories and Research Facilities:**Ensures safe operation by monitoring environmental conditions and electrical parameters.
- **Smart Building Automation:**Integrates with automation systems to control appliances and improve energy efficiency.
- **Electrical Equipment Monitoring:**Prevents damage by detecting overload, voltage fluctuations, and abnormal current flow.

13. Advantages

Benefits Over Traditional or Existing Systems

- **Real-Time Monitoring:**Continuously tracks electrical and environmental parameters, unlike traditional systems that lack live monitoring.
- **Early Fault Detection:**Detects issues such as gas leakage, overheating, and electrical faults at an early stage, reducing risk of accidents.
- **Multi-Parameter Integration:**Combines multiple sensors (gas, temperature, voltage, current, motion, vibration) in a single system.
- **IoT-Based Remote Access:**Enables remote monitoring through the Blynk application from anywhere.

- **Automatic Safety Control:** Activates buzzer and relay automatically when abnormal conditions are detected.
- **Cost-Effective Solution:** Uses low-cost components like Arduino and ESP8266, making it affordable compared to complex systems.
- **Scalable and Flexible:** Additional sensors and features can be easily integrated in the future.
- **Improved Safety and Reliability:** Reduces human intervention and increases system reliability.

14. Limitations

Current Constraints of the Project

- **Limited Accuracy of Sensors:** Low-cost sensors like MQ4 and ACS712 may produce less accurate readings and require frequent calibration.
- **Dependence on Wi-Fi Connectivity:** The system relies on stable internet connection for IoT monitoring through the Blynk platform.
- **Power Supply Constraints:** Continuous operation may be affected due to battery limitations and power consumption of multiple components.
- **No Data Storage:** The system does not store historical data for analysis or future reference.
- **Limited Range of Monitoring:** Designed mainly for small-scale applications like homes or labs, not suitable for large-scale industrial systems.
- **Basic Alert Mechanism:** Alerts are limited to buzzer and app notifications; no SMS or email alert integration.

- **Manual Threshold Setting:** Threshold values are predefined and not adaptive to changing conditions.
- **System Complexity:** Integration of multiple sensors and modules increases wiring complexity and maintenance effort.

15. Future Scope

Possible Enhancements and Improvements

- **Cloud Data Storage and Analytics:**
Integrate cloud platforms to store historical data and perform data analysis for better insights and decision-making.
- **Mobile App Enhancement:**
Improve the user interface and add advanced features in the Blynk app for better user experience.
- **AI-Based Predictive Maintenance:**
Implement machine learning algorithms to predict faults and prevent failures before they occur.
- **Advanced Alert System:**
Add SMS, email, or push notifications for critical alerts to enhance safety.
- **Integration with Smart Grid Systems:**
Connect the system with smart grids for efficient energy distribution and management.
- **Use of High-Precision Sensors:**
Replace basic sensors with industrial-grade sensors for improved accuracy and reliability.
- **Mobile Application Development:**
Develop a dedicated custom mobile app instead of relying only on third-party platforms.

- **Scalability for Industrial Use:**
Expand the system to support large-scale industrial environments with multiple nodes.
- **Automation and Remote Control:**
Enable users to remotely control electrical appliances through IoT.

16. Conclusion

The proposed IoT-Based Smart Energy Monitoring and Electrical Safety Control System successfully integrates multiple sensors to monitor both environmental and electrical parameters in real time. The system effectively detects abnormal conditions such as gas leakage, temperature rise, over-voltage, and motion, ensuring improved safety and reliability.

By combining Arduino UNO for data processing and ESP8266 for IoT communication, the system enables continuous monitoring and remote access through the Blynk application. The implementation of automatic alert mechanisms using a buzzer and relay enhances the system's ability to respond instantly to hazardous situations.

The project demonstrates a cost-effective, scalable, and efficient solution suitable for smart homes, laboratories, and small-scale industrial applications. Overall, the system achieves its objective of improving electrical safety, reducing risks, and enabling smart energy management through IoT technology.

References

Paper 1: Internet-of-Things-Based Smart Home Energy Management System with Multi-Sensor Data Fusion

Link: <https://doi.org/10.3390/engproc2024066010>

Paper 2: Smart Home and Security Systems: An IoT-Based Approach Utilizing ESP 32 and Multi-sensor Integration

Link: <https://doi.org/10.1051/e3sconf/202561602004>

Paper 3: An IoT System for Social Distancing and Emergency Management in Smart Cities Using Multi-Sensor Data

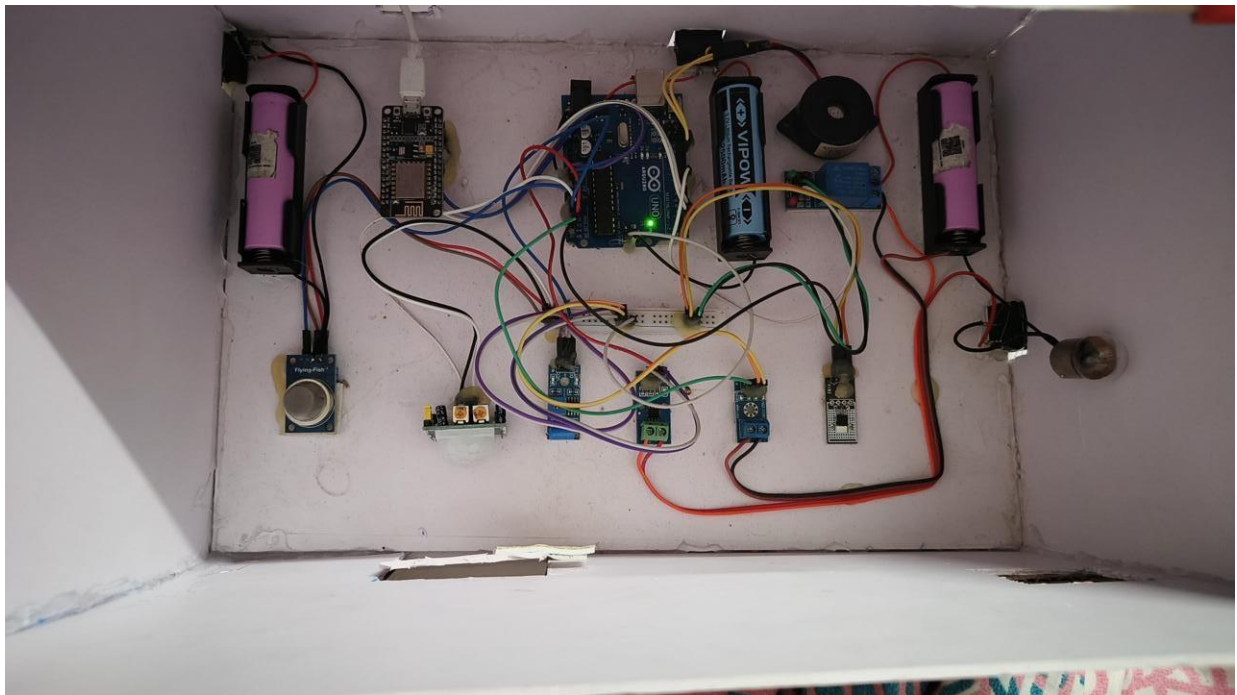
Link: <https://doi.org/10.3390/a13100254>

Paper 4: An IoT-Enabled Multi-Sensor System with Location Detection for Agricultural Applications

Link: <https://link.springer.com/article/10.1007/s12647-022-00617-7>

Appendix:

Additional Circuit Picture:



Full source code:

Code:

```
#define BLYNK_TEMPLATE_ID "TMPL3NN0iLrgp"
#define BLYNK_TEMPLATE_NAME "ESP8266"
#define BLYNK_AUTH_TOKEN "IWNdOLWlgxoulrF5JMY7-
twUTeST0l_4"

#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <SoftwareSerial.h>
#include <BlynkSimpleEsp8266.h>

const char* ssid = "Rishi";
```

```
const char* password = "Rishikeshwar31";
```

```
ESP8266WebServer server(80);
```

```
SoftwareSerial unoSerial(D6, D7);
```

```
BlynkTimer timer;
```

```
#define RELAY D5
```

```
#define BUZZER D4
```

```
float gas=0, pir=0, vib=0, curr=0, volt=0, temp=0, light=0;
```

```
String lastLine = "";
```

```
bool vibAlertSent = false;
```

```
bool powerAlertSent = false;
```

```
int vibCount = 0;
```

```
void setup() {
```

```
    Serial.begin(9600);
```

```
    unoSerial.begin(9600);
```

```
    pinMode(RELAY, OUTPUT);
```

```
    pinMode(BUZZER, OUTPUT);
```

```
digitalWrite(RELAY, HIGH);  
digitalWrite(BUZZER, LOW);
```

```
WiFi.begin(ssid, password);  
Serial.print("Connecting...");  
while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
}
```

```
Serial.println("\nConnected!");  
Serial.println(WiFi.localIP());
```

```
Blynk.begin(BLYNK_AUTH_TOKEN, ssid, password);
```

```
timer.setInterval(1000L, sendDataToBlynk);
```

```
server.on("/", handleRoot);  
server.begin();  
}
```

```
void loop() {  
    Blynk.run();  
    timer.run(); // VERY IMPORTANT  
    server.handleClient();  
}
```

```
if(unoSerial.available()) {  
    String line = unoSerial.readStringUntil('\n');  
    line.trim();  
  
    Serial.println("RAW: " + line);  
  
    if (line.startsWith("GAS:")) {  
        lastLine = line;  
  
        parseData(line);  
        safetyCheck();  
    }  
}  
}  
}  
  
void parseData(String data) {  
    int start = 0;  
  
    while (true) {  
        int commaIndex = data.indexOf(',', start);  
  
        String pair = (commaIndex == -1) ? data.substring(start) :  
data.substring(start, commaIndex);  
  
        int colonIndex = pair.indexOf(':');  
  
        if (colonIndex != -1) {
```

```
String name = pair.substring(0, colonIndex);  
float value = pair.substring(colonIndex + 1).toFloat();
```

```
if (name == "GAS") gas = value;  
if (name == "PIR") pir = value;  
if (name == "VIB") vib = value;  
if (name == "CURR") curr = value;  
if (name == "VOLT") volt = value;  
if (name == "TEMP") temp = value;  
if (name == "LIGHT") light = value;  
}
```

```
if (commaIndex == -1) break;  
start = commaIndex + 1;  
}
```

```
Serial.print("VIB: ");  
Serial.println(vib);  
}
```

```
void sendDataToBlynk() {  
  Blynk.virtualWrite(V0, gas);  
  Blynk.virtualWrite(V1, pir);  
  Blynk.virtualWrite(V2, vib);  
  Blynk.virtualWrite(V3, curr);  
}
```

```
Blynk.virtualWrite(V4, volt);
```

```
Blynk.virtualWrite(V5, temp);
```

```
Blynk.virtualWrite(V6, light);
```

```
if (vibCount > 3 && !vibAlertSent) {
```

```
    Blynk.logEvent("vibration_alert", "Vibration detected!");
```

```
    vibAlertSent = true;
```

```
}
```

```
if (vibCount == 0) vibAlertSent = false;
```

```
if (gas > 140) Blynk.logEvent("gas_alert", " Gas Leak!");
```

```
if (temp > 55) Blynk.logEvent("temp_alert", " Overheat!");
```

```
if (volt > 220) Blynk.logEvent("volt_alert", " High Voltage!");
```

```
if (curr > 4.0) Blynk.logEvent("current_alert", " Overcurrent!");
```

```
}
```

```
void safetyCheck() {
```

```
    if (vib == 0) {
```

```
        vibCount++;
```

```
    } else {
```

```
        vibCount = 0;
```

```
    }
```

```
    bool vibDanger = (vibCount > 3);
```

```
bool otherDanger = (gas > 400 || temp > 55 || curr > 4.0 || volt > 220);
```

```
if (vibDanger) {  
    digitalWrite(BUZZER, HIGH);  
} else {  
    digitalWrite(BUZZER, LOW);  
}
```

```
if (otherDanger) {  
    digitalWrite(RELAY, LOW);
```

```
    if (!powerAlertSent) {  
        Blynk.logEvent("power_cut", Power OFF!");  
        powerAlertSent = true;  
    }  
} else {  
    digitalWrite(RELAY, HIGH);  
    powerAlertSent = false;  
}  
}  
  
BLYNK_WRITE(V7) {  
    int state = param.asInt();  
    digitalWrite(RELAY, state ? HIGH : LOW);  
}
```



```

void handleRoot() {
    String html = "<html><head><title>Dashboard</title>";
    html += "<meta http-equiv='refresh' content='2'>";
    html += "<style>body {text-align:center;} table {margin:auto;border-collapse:collapse;} th,td {border:1px solid #000;padding:8px;}</style>";
    html += "</head><body>";

    html += "<h2>Smart Energy Monitoring</h2>";
    html +=
"<table><tr><th>Sensor</th><th>Value</th><th>Status</th></tr>";

    addRow(html, "GAS", gas, gas > 400);
    addRow(html, "PIR", pir, pir > 0);
    addRow(html, "VIB", vib, vibCount > 3);
    addRow(html, "CURR", curr, curr > 4.0);
    addRow(html, "VOLT", volt, volt > 220);
    addRow(html, "TEMP", temp, temp > 55);
    addRow(html, "LIGHT", light, light == 0);

    html += "</table>";

    html += "<h3>Power: ";
    html += (digitalRead(RELAY) ? "ON" : "OFF");
    html += "</h3>";

```

```
html += "</body></html>";
```

```
server.send(200, "text/html", html);  
}
```

```
// Helper
```

```
void addRow(String &html, String name, float value, bool alert) {  
    html += "<tr><td>" + name + "</td><td>" + String(value) +  
    "</td><td>" + (alert ? "ALERT" : "OK") + "</td></tr>";  
}
```